

# Simple Linear Regression

## Cell-by-Cell Jupyter Notebook Explanation

**Learning goal:** Understand what every cell in the supplied notebook does, why it is used, and how it fits into a machine-learning workflow.

**Source:** Simple Linear Regression(1).ipynb — 43 cells. The explanation below is based on the notebook content itself.

| Stage                             | Notebook cells | Purpose                                             |
|-----------------------------------|----------------|-----------------------------------------------------|
| 1. Data Gathering                 | 1–3            | Import pandas and load 50_Startups.csv              |
| 2. Data Quality                   | 4–8            | Check shape, data types, duplicates, missing values |
| 3. Relationship Analysis          | 9–16           | Scatterplots, correlation, choose X and Y           |
| 4. Feature Engineering / Pipeline | 17–21          | Introduce preprocessing and median imputation       |
| 5. Train/Test Split               | 22–32          | Create training/testing sets and preprocess them    |
| 6. Model Building                 | 33–36          | Train Linear Regression and inspect coefficients    |
| 7. Prediction                     | 37–39          | Predict PROFIT for test data                        |
| 8. Evaluation                     | 40–42          | Calculate MSE, RMSE, MAE and R <sup>2</sup>         |

## Cell 1 — CODE

```
import pandas as pd
```

**What it does:** This imports the **pandas** library and gives it the short name **pd**. Pandas is used for loading and working with tabular data such as CSV files and DataFrames.

## Cell 2 — MARKDOWN

**Step1 : Data Gathering / Data Ingestion**

**Explanation:** This is the notebook's first named stage: **Data Gathering / Data Ingestion**. The purpose is to bring the dataset into the Python environment.

## Cell 3 — CODE

```
import pandas as pd
path = r"E:\DATA SCIENTIST\Analysis\pandas\datasets\50_Startups.csv"
df = pd.read_csv(path)
df.head()
```

**What it does:** The notebook stores the CSV location in **path**, then **pd.read\_csv()** reads the file into a pandas DataFrame called **df**. **df.head()** displays the first five rows so we can quickly inspect the dataset.

## Cell 4 — CODE

```
## Step2 : Perform Basic Data Quality Checks
```

**What it does:** This code cell executes part of the notebook workflow.

## Cell 5 — CODE

```
df.shape
```

**What it does:** **df.shape** returns a tuple in the form **(rows, columns)**. This is a quick way to understand the size of the dataset before modeling.

## Cell 6 — CODE

```
df.info()
```

**What it does:** **df.info()** displays the number of entries, column names, non-null counts, and data types. It helps identify whether columns are numeric/text and whether missing values may exist.

## Cell 7 — CODE

```
# If there are any duplicates
df.duplicated().sum()
```

**What it does:** **df.duplicated()** identifies repeated rows. **.sum()** counts them. Duplicate checking is part of basic data-quality validation.

## Cell 8 — CODE

```
# If there is any missing data
df.isna().sum()
```

**What it does:** **df.isna()** marks missing values as True, and **.sum()** counts missing values in each column. This helps decide whether preprocessing is required.

## Cell 9 — MARKDOWN

**Scatterplot to understand the relationship between target feature and other features**

**Explanation:** This note explains why scatterplots are used: to understand the relationship between the target feature (PROFIT) and other numeric features.

## Cell 10 — CODE

```
import seaborn as sns
import matplotlib.pyplot as plt
cols = ["RND", "ADMIN", "MKT"]
for i in cols:
    sns.scatterplot(data=df, x=i, y="PROFIT")
    plt.show()
```

**What it does:** The notebook imports Seaborn and Matplotlib. It creates a list containing **RND**, **ADMIN**, and **MKT**. The loop creates one scatterplot for each feature against **PROFIT**. Scatterplots help visually inspect whether a roughly linear relationship may exist.

## Cell 11 — MARKDOWN

**Explanation:** This markdown cell is empty and does not add content to the workflow.

## Cell 12 — CODE

```
cols.append('PROFIT')
df[cols].corr()
```

**What it does:** The code adds **PROFIT** to the feature list and calculates the correlation matrix for RND, ADMIN, MKT, and PROFIT. Correlation measures the strength and direction of a linear relationship. A value near +1 indicates strong positive linear association; near 0 indicates weak linear association; near -1 indicates strong negative linear association.

## Cell 13 — MARKDOWN

**x can be MKT or RND**

**Explanation:** The notebook notes that X can be MKT or RND. The later code selects RND, so the executed workflow uses RND as the single predictor.

## Cell 14 — CODE

```
x= df[['RND']]
y = df[['PROFIT']]
```

**What it does:** **x** is the input feature and is selected as the **RND** column. Double brackets keep X as a two-dimensional DataFrame, which is the expected feature format for scikit-learn. **y** is the target, **PROFIT**. The notebook's markdown says X can be MKT or RND, but this actual code chooses RND.

## Cell 15 — CODE

```
x.head()
```

**What it does:** **x.head()** shows the first five values of the selected input feature. This is a simple verification step before continuing.

## Cell 16 — CODE

```
y.head()
```

**What it does:** **y.head()** shows the first five target values. It confirms that PROFIT has been selected as the prediction target.

## Cell 17 — MARKDOWN

**## Feature Engineering - Adding of columns,datatype transformations,data cleaning(wrangling),data processing, curse of dimensiona**

**\*\*Data Preprocessing: Data cleaning\*\***

**Explanation:** This section introduces feature engineering and data preprocessing. It describes adding columns, datatype transformations, cleaning/wrangling, processing, and feature-selection/dimensionality concepts. The actual pipeline used later is focused on missing-value preprocessing.

## Cell 18 — CODE

```
x.head()
```

**What it does:** The notebook displays `x.head()` again. This cell does not transform the data; it simply verifies the feature values before the pipeline stage.

## Cell 19 — MARKDOWN

```
## PipeLine
```

**Explanation:** This heading introduces the preprocessing pipeline stage.

## Cell 20 — CODE

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer

num_pipe = make_pipeline(SimpleImputer(strategy='median')).set_output(transform='pandas')
num_pipe
```

**What it does:** The notebook imports `make_pipeline` and `SimpleImputer`. The pipeline uses median imputation, meaning missing numeric values would be replaced by the median of the training feature.

`set_output(transform='pandas')` asks scikit-learn to return pandas output after transformation. In this notebook, the pipeline contains only the imputation step.

## Cell 21 — MARKDOWN

**\*\*In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook. On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.\*\***

**Explanation:** This is an informational Jupyter/GitHub rendering note. It does not perform model training or data transformation.

## Cell 22 — MARKDOWN

```
## Step 4: Split the data into training and testing
```

```
training: 80%
testing: 20%
```

```
fit() : fit on training data
fit_transform() : fit on training data
transform(): on testing data
```

**Explanation:** This section explains the intended train/test split: 80% training and 20% testing. It also distinguishes `fit()`, `fit_transform()`, and `transform()`.

## Cell 23 — CODE

```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest = train_test_split(x,y,train_size=0.8,test_size=0.2,random_state=21)
```

**What it does:** `train_test_split()` separates X and y into training and testing portions. The notebook uses 80% for training and 20% for testing. `random_state=21` makes the split reproducible, so the same random split can be obtained again.

## Cell 24 — CODE

```
xtrain.head()
```

**What it does:** `xtrain.head()` displays sample rows from the training input data.

## Cell 25 — CODE

```
xtest.head()
```

**What it does:** `xtest.head()` displays sample rows from the testing input data.

## Cell 26 — CODE

```
ytrain.head()
```

**What it does:** `ytrain.head()` displays sample target values used for model training.

## Cell 27 — CODE

```
ytest.head()
```

**What it does:** `ytest.head()` displays sample target values reserved for evaluating the model.

## Cell 28 — CODE

```
## to print x test and train and also y test and train
print("X Training and Testing :", xtrain.shape , xtest.shape)
print("Y Training and Testing :", ytrain.shape , ytest.shape)
```

**What it does:** The code prints the shapes of X training/testing and y training/testing. This confirms that the split produced the expected sizes and that the input and target partitions align.

## Cell 29 — CODE

```
num_pipe.fit(xtrain)
```

**What it does:** `num_pipe.fit(xtrain)` learns the preprocessing information from the training data. For median imputation, this means learning the median from the training feature. Importantly, the preprocessing is fitted only on training data to avoid leaking information from the test set.

## Cell 30 — MARKDOWN

**\*\*In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.  
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.\*\***

**Explanation:** This is another Jupyter/GitHub rendering note. It does not itself perform a modeling operation.

## Cell 31 — CODE

```
xtrain_pre = num_pipe.transform(xtrain)
xtest_pre = num_pipe.transform(xtest)
```

**What it does:** The fitted pipeline transforms both `xtrain` and `xtest`. Because the pipeline was fitted on training data, the same learned transformation is applied to the test data.

## Cell 32 — CODE

```
xtrain.head()
```

**What it does:** The notebook calls `xtrain.head()` again. Notice that this displays the original training DataFrame rather than the transformed `xtrain_pre` variable.

## Cell 33 — MARKDOWN

**## Step 5: Model Building**

**Linear Regression: X=RND, Y = PROFIT**

**$Y_{preds} = RND \times B1 + B0$**

**$B0 \rightarrow$  yintercept : value of Y when RND is 0.**

**Explanation:** This section defines the model concept used by the notebook:  $X = RND$  and  $Y = PROFIT$ . The stated equation is  $Y_{pred} = RND \times B1 + B0$ , where  $B1$  is the slope and  $B0$  is the intercept.

## Cell 34 — CODE

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(xtrain_pre,ytrain)
```

**What it does:** The notebook imports **LinearRegression**, creates a model object, and calls **fit()** using the preprocessed training feature and training target. The model learns the best-fitting straight-line relationship between RND and PROFIT from the training data.

## Cell 35 — CODE

```
print(f"Yintercept: {model.intercept_},Slope: {model.coef_}")
```

**What it does:** **model.intercept\_** is the y-intercept: the predicted target value when RND is zero. **model.coef\_** contains the slope/coefficient for RND. Together they define the fitted line.

## Cell 36 — MARKDOWN

**Ypredictions = RND \*(0.85) + 49320.11**

**When RND is 0, Ypredictions = 49320.11**

**When RND is 1000, Ypreds = 1000\*0.85 + 49320.11 = 850 + 49320.11**

**Explanation:** The notebook gives an example using the stated equation: **Ypred = RND × 0.85 + 49320.11**. It then demonstrates the interpretation when RND is 0 and 1000. These are the values written in the notebook's markdown, not values independently recalculated from the CSV here.

## Cell 37 — MARKDOWN

**## Predictions**

**Explanation:** This heading introduces the prediction stage.

## Cell 38 — CODE

```
ypred_test = model.predict(xtest_pre)
ypred_test[:5]
```

**What it does:** **model.predict(xtest\_pre)** uses the trained model to predict PROFIT for the unseen test inputs. The first five predictions are displayed using **[:5]**.

## Cell 39 — CODE

```
ytest.head()
```

**What it does:** **ytest.head()** displays actual PROFIT values for the test rows. These actual values can be compared with the predictions from the previous cell.

## Cell 40 — MARKDOWN

**## Performance Metrics: MOdel Evaluation using MSE,MAE,RMSE,R2 score**

**Explanation:** This heading introduces model evaluation using MSE, MAE, RMSE, and R<sup>2</sup>.

## Cell 41 — CODE

```
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score

mse = mean_squared_error(ytest,ypred_test)
rmse = mse**(1/2)
mae = mean_absolute_error(ytest,ypred_test)
r2 = r2_score(ytest,ypred_test)

print(f"MSE : {mse}")
print(f"RMSE : {rmse}")
print(f"MAE : {mae}")
print(f"R2 : {r2*100:.2f}%")
```

**What it does:** The notebook calculates four evaluation metrics. **MSE** is the mean squared error, which squares prediction errors before averaging. **RMSE** is the square root of MSE and is expressed in the target's units. **MAE** is the mean absolute error and represents average absolute prediction error. **R<sup>2</sup>** measures how much variance in the target is explained by the model, relative to a baseline based on the target mean. The notebook prints R<sup>2</sup> as a percentage by multiplying it by 100.

## Cell 42 — MARKDOWN

**## As the R2 score is more than 80%, we can consider this model for final predictions and deployment**

**Explanation:** The notebook states that an R<sup>2</sup> above 80% can be considered suitable for final predictions and deployment. This is the notebook author's stated rule; the notebook does not provide a broader validation or deployment analysis.

## Cell 43 — CODE

**What it does:** This cell contains no code, so it has no execution effect.

# ML Developer View — Complete Workflow

| Step | Stage             | What happens                                               |
|------|-------------------|------------------------------------------------------------|
| 1    | Data Ingestion    | Load 50_Startups.csv into DataFrame df.                    |
| 2    | Data Quality      | Check shape, structure, duplicates, and missing values.    |
| 3    | EDA               | Use scatterplots and correlation to inspect relationships. |
| 4    | Feature Selection | Use RND as X and PROFIT as y.                              |
| 5    | Preprocessing     | Create a median-imputation pipeline.                       |
| 6    | Train/Test Split  | Use 80% training and 20% testing with random_state=21.     |
| 7    | Fit Preprocessor  | Learn imputation information from training data only.      |
| 8    | Model Training    | Train LinearRegression on RND → PROFIT.                    |
| 9    | Prediction        | Predict PROFIT on the test set.                            |
| 10   | Evaluation        | Calculate MSE, RMSE, MAE, and R <sup>2</sup> .             |

## Important notebook-specific observations

- The notebook uses RND as the actual single input feature, even though its markdown says X can be MKT or RND.
- The preprocessing pipeline contains only SimpleImputer(strategy='median'). No scaling or encoding step is present.
- The pipeline is fitted on xtrain and then used to transform both xtrain and xtest, which follows the intended train/test preprocessing pattern.
- The exact printed MSE, RMSE, MAE, and R<sup>2</sup> values are not stored in the notebook source shown here, so this PDF does not invent those numbers.
- The markdown example uses an intercept of 49320.11 and slope of 0.85. These values are presented as the notebook's example; they are not independently verified against the CSV file path because that path is a local Windows path from the notebook.
- The final cell is empty.

## Key concepts to remember

- **Feature (X):** input used by the model to make a prediction. Here: RND.
- **Target (y):** value the model tries to predict. Here: PROFIT.
- **Linear Regression:** learns a straight-line relationship between X and y.
- **Slope:** expected change in predicted PROFIT for a one-unit increase in RND, according to the fitted model.
- **Intercept:** predicted PROFIT when RND equals zero.
- **Training data:** data used to learn model parameters.
- **Testing data:** unseen data used to evaluate how the trained model performs.
- **MAE:** average absolute prediction error.
- **MSE:** average squared prediction error.
- **RMSE:** square root of MSE, in target units.

- **$R^2$** : proportion of target variance explained by the regression model relative to the mean baseline.

**Study sequence:** First understand each cell, then understand the complete pipeline, and finally practice explaining why each ML step is needed in an interview.